S6 — Reranking: A Full Worked Example

File: `src/rerank.py`

This document explains exactly how reranking works, using a real session from the public set (`public_0008`) with actual catalog data and numbers that match the observed trace scores precisely.

Where `product["text"]` Comes From

Every product in the catalog (`data/catalog.jsonl`) is a JSON object with fields like `title`, `features`, `description`, and `details`. During the one-time index build (`src/index.py`, `CatalogIndex._build()`), a trimmed record is created for each product and stored in `index.products[parent_asin]`. One of its fields is `"text"`:

```
# src/index.py  (lines 76-78)
"text": " ".join(
    TOKEN_RE.findall(f"{title} {features} {description} {flatten(details)}")
).lower(),
```

It concatenates `title + features + description + details`, strips everything that is not a letter or digit (punctuation, `%`, `;`, `/` etc.), joins the tokens with spaces, and lowercases the result. The purpose of this normalisation is to make substring matching punctuation-insensitive: `"96% Nylon, 4% Spandex"` becomes `"96 nylon 4 spandex"`, and `"Pull-On closure"` becomes `"pull on closure"`. The customer's disclosed spans are normalised the same way (`src/text.py`, `constraint_spans()`), so they match.

`"text"` does **not** include the `categories` or `store` fields — those live only in the FTS5 index for retrieval, not in the reranker.

The Session: `public_0008`

Target product: `B0BPCC1KBT` — Hanes Womens Wireless Bra, Full-Coverage Pullover Stretch-Knit Bra. **Price:** $10.99. **Rating:** 4.0/5. **Reviews:** 30,628.

The raw catalog `features` for the target (this is what the evaluator copied from to build the customer's messages):

```
"features": [
    "96% Nylon, 4% Spandex",
    "Pull-On closure",
    "Hand Wash Only",
    "THE SUPPORT YOU NEED IS KNIT RIGHT IN - Strategic knit panels...",
    ...
]
```

After `_build()` processes this, `index.products["B0BPCC1KBT"]["text"]` begins:

```
hanes womens wireless bra full coverage pullover stretch knit bra smoothing
t shirt bra 96 nylon 4 spandex pull on closure hand wash only the support
you need is knit right in strategic knit panels and a ribbed underband ...
```

Notice: `"96% Nylon, 4% Spandex"` → `"96 nylon 4 spandex"` and `"Pull-On closure"` → `"pull on closure"` and `"Hand Wash Only"` → `"hand wash only"`.

The Conversation (Turns 1–3)

```
Turn 1  Customer: "I'm looking for Bras Everyday Bras. A key requirement is: nylon."
        Agent   : asks 'other'  |  list withheld

Turn 2  Customer: "For that, what matters is: 96% Nylon, 4% Spandex; Pull-On closure."
        Agent   : asks 'other'  |  list withheld
        [revealed: "96% Nylon, 4% Spandex"  and  "Pull-On closure"]

Turn 3  Customer: "For that, what matters is: Hand Wash Only."
        Agent   : shows 10 products  →  TARGET at rank 1
        [revealed: "Hand Wash Only"]
```

Step 1 — Collecting the Constraint Spans

`state.query_spans()` collects every multi-word phrase from turn 2 onwards (turn 1 is the agent's framing, not product copy). Each phrase is split on punctuation and normalised by `constraint_spans()`: strip punctuation, lowercase, token-join.

After turns 2 and 3, the four spans in play are:

```
'96 nylon'          (from "96% Nylon, 4% Spandex" - first chunk)
'4 spandex'         (from "96% Nylon, 4% Spandex" - second chunk)
'pull on closure'   (from "Pull-On closure")
'hand wash only'    (from "Hand Wash Only")
```

Note: `"nylon"` from turn 1 is a single-word span — `constraint_spans()` requires `min_words=2`, so single words are excluded from span matching (but still feed BM25).

Step 2 — The Candidate Pool After Retrieval

At turn 3, BM25 retrieval returned 300 candidates. The target was at **pool rank 1** (highest BM25 score). Two strong competitors were at ranks 2 and 3 — all three are nylon/spandex bras with pull-on closure, retrieved for the same reasons.

Pool rank	ASIN	Title
1	`B0BPCC1KBT`	Hanes Womens Wireless Bra (**TARGET**)
2	`B08ML8YGDZ`	ohlyah Women's Sports Bra Seamless Comfort
3	`B075V115P6`	Pretty Seamless 3-Pack Seamless Wireless Bra

The reranker looks at the **top 200** (`config.depth = 200`) and rescores them.

Step 3 — The `"text"` Field for Each Competitor

TARGET `B0BPCC1KBT` (first 180 chars of text):

`hanes womens wireless bra full coverage pullover stretch knit bra smoothing t shirt bra 96 nylon 4 spandex pull on closure hand wash only ...`

Competitor `B08ML8YGDZ` — ohlyah Sports Bra (first 180 chars):

`ohlyah women s sports bra seamless comfort yoga bras with removable pads 96 nylon 4 spandex imported pull on closure innovative non wired ...`

Competitor `B075V115P6` — Pretty Seamless 3-Pack (first 180 chars):

`pretty seamless 3 pack women s seamless wireless strappy cage front bustier 96 nylon 4 spandex imported pull on closure machine wash sexy spaghetti ...`

Step 4 — Span Matching: `if span in text`

The reranker does a plain Python substring check for each span:

```python
# src/rerank.py (lines 76-79)
coverage = 0.0
for span in spans:
    if span in text:
        coverage += 1.0 + config.length_bonus * len(span.split())
```

`length_bonus = 0.12`: each word in the span adds 0.12 extra. A 2-word span scores `1.0 + 0.24 = 1.24`. A 3-word span scores `1.0 + 0.36 = 1.36`.

TARGET `B0BPCC1KBT`

Span	Words	In `text`?	Contribution
`'96 nylon'`	2	yes	`1.0 + 0.12×2 = 1.240`
`'4 spandex'`	2	yes	`1.0 + 0.12×2 = 1.240`
`'pull on closure'`	3	yes	`1.0 + 0.12×3 = 1.360`
`'hand wash only'`	3	yes	`1.0 + 0.12×3 = 1.360`
coverage total			**5.200**

All four match: all four were copied verbatim from this product's own features list.

Competitor `B08ML8YGDZ` (ohlyah Sports Bra)

Span	Words	In `text`?	Contribution
`'96 nylon'`	2	yes	`1.240`
`'4 spandex'`	2	yes	`1.240`
`'pull on closure'`	3	yes	`1.360`
`'hand wash only'`	3	**no**	`0.000`
coverage total			**3.840**

The ohlyah bra does not say "Hand Wash Only" anywhere (it says "Please take out the removable pads before washing" — no shared tokens with `hand wash only`).

Competitor `B075V115P6` (Pretty Seamless 3-Pack)

Span	Words	In `text`?	Contribution
`'96 nylon'`	2	yes	1.240
`'4 spandex'`	2	yes	1.240
`'pull on closure'`	3	yes	1.360
`'hand wash only'`	3	**no**	0.000
coverage total			**3.840**

The Pretty Seamless bra says "MACHINE WASH COLD" — the opposite of hand wash.

Step 5 — Combining the Three Signals

```
# src/rerank.py (lines 80-84)
total = (
    config.span_weight      * coverage
  + config.retrieval_weight * (retrieval_score / top_score)
  + config.popularity_weight * _popularity(product)
)
```

Signal 1: span coverage (weight `1.0`) — computed above.

Signal 2: normalised retrieval score (weight `1.0`) — BM25 score divided by the top BM25 score in the pool, placing it in `[0, 1]`. The target held pool rank 1, so it gets `1.00`. Competitors were just below.

Signal 3: popularity (weight `0.02`) — formula from `_popularity()`:

```
(average_rating / 5.0) * min(1.0, log10(rating_number + 1) / 4.0)
```

Maximum possible contribution is `0.02 × 1.0 = 0.020`. It is a tie-break only.

TARGET `B0BPCC1KBT` — rating=4.0, reviews=30,628

```
popularity = (4.0/5.0) × min(1.0, log10(30629)/4) = 0.800 × 1.0 = 0.8000

total = 1.0 × 5.2000   (span coverage - 4 spans matched)
      + 1.0 × 1.0000   (BM25 normalised, pool rank 1)
      + 0.02 × 0.8000  (popularity)
      = 6.2160
```

Competitor `B08ML8YGDZ` — rating=4.1, reviews=954

```
popularity = (4.1/5.0) × (log10(955)/4) = 0.820 × 0.745 = 0.6109

total = 1.0 × 3.8400   (span coverage - only 3 spans matched)
      + 1.0 × 0.9500   (BM25 normalised, pool rank ~2)
      + 0.02 × 0.6109  (popularity)
      = 4.8022
```

Competitor B075V115P6 — rating=3.6, reviews=159

```
popularity = (3.6/5.0) × (log10(160)/4) = 0.720 × 0.551 = 0.3967

total = 1.0 × 3.8400   (span coverage - only 3 spans matched)
      + 1.0 × 0.9300   (BM25 normalised, pool rank ~3)
      + 0.02 × 0.3967  (popularity)
      = 4.7779
```

Step 6 — The Final Ranking

```
Rank 1  B0BPCC1KBT  score=6.2160  Hanes Womens Wireless Bra       <- TARGET
Rank 2  B08ML8YGDZ  score=4.8022  ohlyah Women's Sports Bra
Rank 3  B075V115P6  score=4.7779  Pretty Seamless 3-Pack Bra
```

These match the trace in `runs/public-20260829-123201/sessions/public_0008.md` exactly (the trace records `6.216` for the target). The target wins by **1.41 points** over the nearest competitor, entirely on the strength of one phrase: `'hand wash only'`. That single 3-word span contributed `1.360` to the target and `0.000` to either competitor.

Why Popularity Weight Is 0.02

The Hanes bra has **30,628 reviews** — far more than either competitor. Yet its popularity contribution (`0.0160`) barely differs from ohlyah's (`0.0122`). That is by design. If popularity weight were `1.0`, a product with ten times the reviews would score ~1 point higher regardless of matching the customer's constraints. The target is one specific person's purchase, not a bestseller. Popularity is a tie-break only.

Summary

```
Pool of 300 from retrieval
         |
         v
Take top 200 only  (tail is too weak to rerank)
         |
         v
For each of 200 candidates:

  1. Read index.products[asin]["text"]
       = token-joined lowercase string of title+features+description+details
       = built once at startup from catalog.jsonl

  2. For each disclosed span (customer's exact phrases, same normalisation):
       if span in text:
           coverage += 1.0 + 0.12 * word_count

  3. total = 1.0*coverage + 1.0*(bm25/top_bm25) + 0.02*popularity
         |
         v
```

```
Sort top 200 by total score, append untouched tail
         |
         v
New ordered pool of 300  ->  S7 timing decides what to show
```

The key: `product["text"]` and the customer's spans are normalised identically, so verbatim substring matching is reliable. It works because the simulated customer's constraints are copied from the target's own metadata — the same source that `product["text"]` was built from at index time.